

Applied Engineering Optimisation

ENEL445-24S1

Week 2: Project introduction

- As before, groups of 1-3 students per project
- Larger group = expected to go a bit deeper especially in technical design, so there should be no advantage or disadvantage in either a smaller or larger group.

Today's lecture

- Project description and details
- How to apply optimisation to computationally expensive or intractable problems (**test material** – even if you are not doing this project, you need to know this part)

Jeremy's project: power system optimisation

- See the course reader (Chapter 6) for full details
- See LEARN for example code to get started.



Problem:

- You are given \$10 billion to build generation and/or storage via a mix of:
 - Wind
 - Solar
 - Hydro
 - Geothermal
 - Batteries
- After one year, you aim to maximize your **cash-at-hand**.

Generation

- The costs of each generation method is randomized at the start of the year, and given to your program which decides how much to build.
- Wind and solar are subjected to random factors, in the solar case, based on the time of day.
- Hydro is limited by the amount of water available.
- Geothermal is limited only by its capacity.
- Batteries have a 97.5% one-way efficiency. No degradation is modelled.

Market

- When you supply a randomly changing load, you are paid for energy supplied at the market price (randomly varies around about \$150 / MWh)
- When you fail to supply the load, you are charged for the unsupplied energy.
 - If the shortfall is small, you may be able to buy energy at the market rate.
 - If the shortfall (between generation and load) is too large, you will be charged at a much higher penalty cost (\$1000 / MWh).

Optimisation problem

- Decision variables: everything you control
- Objective function: cash at the end of the year
- Constraints:
 - starting funds (build is limited by this)
 - at each time-step, the possible generation is limited by capacity
 - Some multi-timestep constraints, e.g. if you use all the battery's stored energy in one time-step, you won't be able to get anything from the battery until you charge it again.
- There are actually tens of thousands of decision variables (plus the objective function is very computationally expensive), so the problem cannot be solved directly.
 - need to think about how to break the problem down.

Deliverables: project report (week 7)

- The progress report should cover the following:
 - a little background / intuition about the problem;
 - a short overview of possible methods to use for the problem;
 - a plan for your solution;
 - any progress-to-date.
- It is very easy to find a solution, even by trial and error, better than the given starting solution, so it is expected to show some preliminary improvement by the time of the progress report.

Deliverables: oral presentation (Week 11)

- Regular lecture times will be dedicated to oral presentations. By then you should have your technical solution.
- Focus on your use of optimisation – don't waste too much time explaining the problem (many of us will already understand the problem, what's interesting is your solution).
- Graphs and figures are helpful to illustrate your results. State the value of the objective function you achieved.
- Try to use best practice for oral presentations – as taught in ENEL400 lectures.

Deliverables – final report (week 12)

- **Presentation:** Reports should be tidy and concise. Use graphs / figures as appropriate to illustrate your solution. IEEE format (<https://www.ieee.org/conferences/publishing/templates.html>) is suggested.
- **Quality of the technical design:** Explain what methods you tried and justify your final choice. Solutions which attempt to optimize both the build and operation process are encouraged, and should demonstrate benefit compared to only optimizing the build using the provided operation code. Comparisons of different optimisation approaches are encouraged.

Final report marking

Item	Weight
Presentation	0.25
Quality of the technical design	0.50
Performance	0.25

Table 6.2: Final report marking

Performance marking

- The best solution from the class will score full marks for performance, so long as it achieves a score of \$6.5 billion or better.
- Marks thereafter will be awarded linearly based on the average score with the class median set at 70%;
 - all complete and working solutions that are better than the starting one will score at least 30% for this section – can be achieved very easily.
 - I may adjust marks **upwards** if the mark defined above seems unfair due to peculiarities of the distribution. But you will not receive a lower mark than what I've defined above.
- Bonus marks of 10% (for this section) will be awarded for any solution which outperforms my own sub-optimal solution, which will be published on LEARN after the final report due date.
 - If this results in a mark exceeding 100%, the mark will not be capped.

Deliverables: code

- Email your code to me or upload to the LEARN dropbox.
- Please don't share code with other groups. If you want to collaborate, be part of the same group.
- Code quality is not directly marked – however good comments and coding may help me understand your technical solution better.

Quick demo

goto MATLAB

Hint 1

- Not all the starting funds need to be used building generation and/or storage.
- Since the objective is to maximize cash at the end of one year, it may be beneficial to under-invest.
- Any unused starting funds will be added to your cash flow at the end to make your final score.

Hints 2 & 3

- Because there are penalties for failing to supply load (as in the real world), it may be beneficial to have a mix of generation and/or storage.
- The ideal mix of generation and/or storage is likely to depend on the loading, costs, and capacity factors which are randomized each year-long simulation. You may wish to adjust your build plans based on this info which is passed to *build_plants()*, as opposed to sticking to fixed proportions per the demo solution.

Questions?

- If you find problems with the code or if anything is unclear, please contact me.
- I hope you all have fun!!

Things to think about

- When applying optimisation in real-life, very often (as in this case) the problem is computationally intractable and simplifications need to be made
- Actually, this is often the main aspect of applying optimisation. Solver choice etc. is important but more often the key is **achieving a computationally tractable problem.**

Intractability

- A problem is intractable if
 - It is too large to be solved by a brute-force approach,
AND
 - There exist no efficient algorithms to solve the problem
- E.g. factoring a number into primes – you can check each possible factor (2, 3, 5, 7, 11, 13, etc.) i.e. brute-force search but this is not tractable for a large number.

Practical techniques for computationally intractable problems:

- All the techniques we will look at are not guaranteed to work in any or every case, but are presented to give some ideas.
- Some of the ideas presented might work for the project we've been discussing, but **not necessarily**.
- Most of them result in a loss of true optimality, but this is usually okay.
- Need to start by working out why the problem is computationally intractable. (Possibly multiple reasons)

Reason: objective function is too computationally expensive

- Possible solutions:
 - Replace objective function with an approximation of the same
 - Example: 1000 repetitions with randomized parameters; can we get away with a smaller number? Can we even replace each random variable with its mean expectation and only run once?
 - Example 2: objective has terms that are orders of magnitude smaller than the function as a whole. Delete these?
 - Example 3: objective function has computationally expensive functions, e.g. $\sum_{i=1}^{1000} \left(\sin \frac{ix}{10000}\right)^{\frac{i}{10000}}$. Can we replace with a linear, polynomial, or piecewise linear approximation?
 - Sometimes the problem is not formulated well and has objectives which should be posed as constraints. A helpful rule of thumb is to ask yourself if improving that metric indefinitely is desirable or not.

Reason: constraint is too computationally expensive

- Possible solutions:
 - Is the constraint active? If not, maybe we can ignore it (i.e. solve the problem without that constraint). This constraint is called *relaxation*.
 - (Similar to the last slide): Approximate the constraints by less expensive functions.

Reason: problem size is too large

- Possible solutions:

- Split into several sub-problems. How to do this? Where decision variables are weakly coupled or not coupled at all, they can be solved separately.

- Example:

$$\min_{x,y} x^2 + |y - 4|$$

can be solved as two problems:

$$\min_x x^2, \min_y |y - 4|$$

- Example 2: Many problems involving multiple time-steps can be solved separately for each time-step.

Important note: Constraints can couple design variables too!

Reason: problem size is too large

- Possible solutions:
 - Once split into sub-problems, each sub-problem can either be solved once, or one could iterate and hope for convergence: i.e. solve one sub-problem, plug in the new best guess of x into the second sub-problem, etc.
 - Another possible solution is to use evolutionary algorithms. These are not guaranteed to converge but are sometimes able to handle large problems better (no need to calculate the gradient).

Reason: problem size is too large

- Possible solutions:
 - A large intractable problem is often intractable because of scaling behaviour, e.g. $O(n^3)$ complexity. A solution then can be to use heuristic algorithms for a (possibly sub-optimal) solution.
 - Examples: greedy algorithm, uniform cost search, A*, iterative deepening, depth-first search, best-first search: active area of research

Reason: solver is taking too long to converge / is not converging

- A common reason and one of the hardest to solve.
- If not because of size/expensive functions, likely because of:
 - **non-convex** behaviour in objective function or constraints. Non-smooth functions in particular are prone to convergence issues – e.g. gradient methods may (will) fail.
 - (poor) **scaling** is another common cause of convergence issues.

Non-convex functions

- **Fact:** all convex optimisation problems can be solved exactly simply by following the gradient!
- Why are non-convex problems difficult?

Scaling

- As briefly mentioned by Le, scaling is important.
- Example:

Reason: solver is taking too long to converge / is not converging

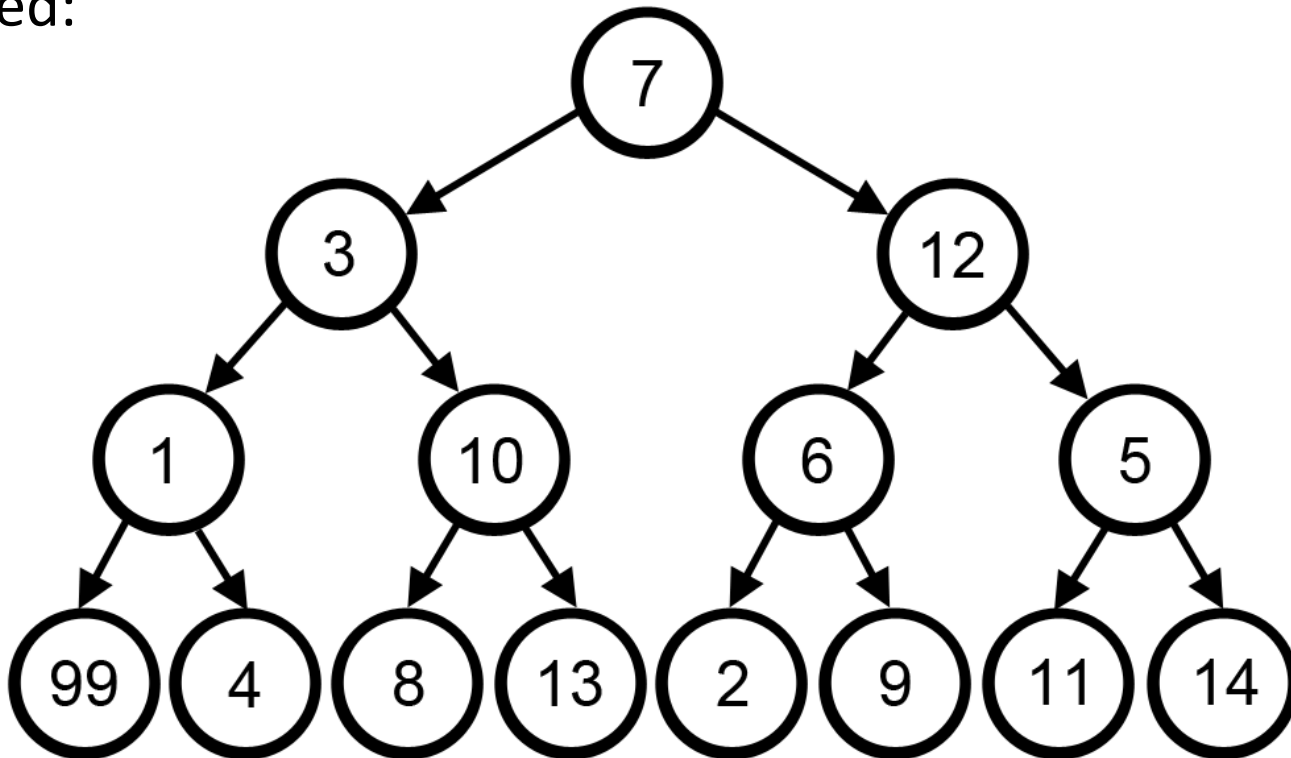
- Possible solutions:
 - Linearise/convexify/relax objective and/or constraint terms which are non-convex
 - Check the scaling.
 - Solve the Lagrangian dual problem (not covered in ENEL445) and check that the duality gap is zero or small
 - Switch to an evolutionary algorithm which doesn't use the gradient
 - Reduce the tolerance, adjust the step-size, etc.
 - Use heuristics as before, e.g. greedy algorithm, UCS, A^* , etc.

Heuristic algorithms

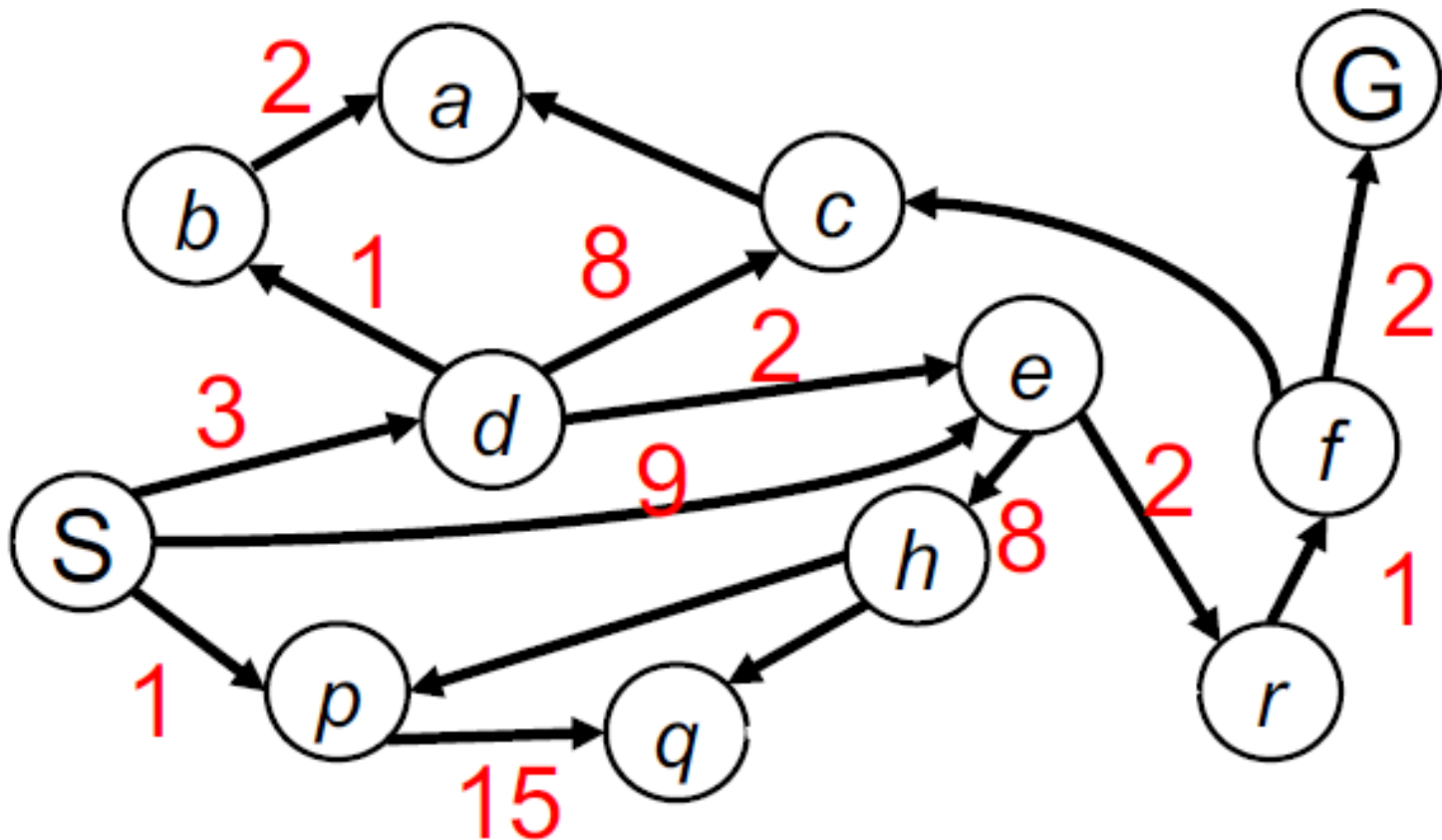
- These are ways to find a solution efficiently (as opposed to brute-force) but usually trade off optimality, completeness, accuracy, or precision for speed.
- A heuristic function, also simply called a heuristic, is a function that ranks alternatives in search algorithms at each branching step based on available information to decide which branch to follow.

Example: Greedy algorithm

- Treat the problem iteratively. At each iteration, take the option that provides the best immediate reward (or lowest immediate cost). Example: find path to maximize numbers visited:



Another example: min cost travelling from S to G



Uniform cost search

- Treat the problem iteratively. At each iteration, take the best rewarded (or lowest cost) path *from the start node*:

